



Design and Implementation of an Autonomous Line-Following Robot for NERC 2026

Yazdan Ali Khan

Student ID: 2024665

Department of Electrical Engineering

Ghulam Ishaq Institute of Engineering and Technology

Email: u2024665@giki.edu.pk

Muhammad Hammad Shahid

Student ID: 2024389

Department of Electrical Engineering

Ghulam Ishaq Institute of Engineering and Technology

Raiyaan Qureshi

Student ID: 2024531

I. ABSTRACT

This paper presents the design and implementation of an autonomous line-following robot developed for the NERC 2026 Autonomous Robot Challenge. The system was built around a PIC18F4550 microcontroller operating at 8 MHz and employed an interrupt-driven state machine architecture to manage three distinct operating modes: standby/calibration, diagnostic drive, and full autonomous operation. The robot utilized five TCRT5000 infrared sensors for line tracking, a TCS3200 colour sensor for pallet sorting, and an L298N H-bridge motor driver for differential steering. A software-generated PWM signal controlled a hobby servo for pallet deployment. The implementation successfully demonstrated autonomous navigation through a six-node course including stations, zigzag sections, ramp traversal, and precision parking. The modular architecture enabled systematic testing and debugging through mode-isolated functionality, while power domain isolation and comprehensive decoupling ensured reliable operation in electrically noisy competition environments.

II. INTRODUCTION

Autonomous mobile robotics competitions serve as valuable platforms for applying embedded systems theory to real-world engineering challenges. The NERC/CEP 2026 Autonomous Robot Challenge required participants to design and construct a line-following robot capable of navigating a complex course while performing colour-discriminated pallet placement tasks at designated stations. The competition course comprised multiple navigation challenges including sharp turns, zigzag sections, an inclined ramp, and precision parking zones, demanding robust sensor fusion and reliable state management.

The primary objective of this project was to develop a competition-ready autonomous robot that could reliably complete the prescribed mission profile while maintaining deterministic behaviour under varying environmental conditions. The robot was required to follow a black line on a white surface, detect course nodes at intersections, execute colour-based sorting operations at three stations, and ultimately park within a defined end zone. Secondary objectives included implementing a diagnostic framework for rapid troubleshooting during competition setup and providing calibration capabilities for sensor alignment.

To address these requirements, an interrupt-driven state machine architecture was selected as the foundational software paradigm. This architectural choice was motivated by several factors. First, the need for instantaneous mode switching via external interrupt ensured that operators could transition between operational states without waiting for polling loops to complete. Second, the state machine abstraction provided clear separation between standby, diagnostic, and autonomous modes, simplifying testing and reducing the

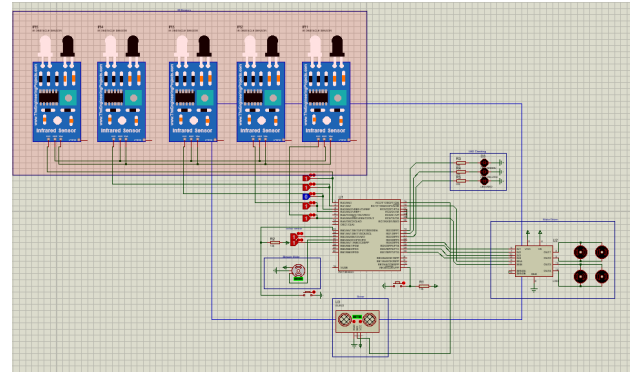
cognitive load during debugging. Third, the use of a dedicated interrupt service routine (ISR) for mode management guaranteed that safety-critical transitions, such as emergency stops, could be executed with minimal latency regardless of the current execution context.

The implementation was realized using a PIC18F4550 microcontroller programmed in C using the XC8 compiler version 3.10 within the MPLAB X integrated development environment version 5.35. The complete system was designed as a two-layer printed circuit board fabricated through commercial PCB services, integrating motor control, sensor interfaces, and power management into a compact, competition-ready platform.

III. SYSTEM OVERVIEW

The robot operated in three distinct modes, each serving a specific role in the development, testing, and competition lifecycle. Mode transitions were managed through a single push-button connected to the RB0/INT0 external interrupt pin, allowing operators to cycle through operational states during setup and competition preparation.

Mode 0, designated as Standby/Calibration mode, served as the default power-on state and provided a safe condition for system initialization and sensor calibration. Upon entering Mode 0, the robot remained stationary with motors disabled while continuously sampling and displaying sensor readings through visual indicators. This mode was essential during competition setup, allowing operators to verify sensor alignment over the line and adjust mechanical positioning without risk of unintended motion. The calibration phase enabled fine-tuning of sensor thresholds and verification of the five-sensor array's response to the track surface before committing to autonomous operation.



Mode 1, Diagnostic Drive mode, provided manual verification of motor wiring and mechanical integrity. In this mode, the robot executed a predetermined test sequence consisting of forward motion, rotational turns in both directions, and gradual curve maneuvers. This systematic test pattern allowed operators to confirm correct motor phase connections, verify that the H-bridge driver was responding to control signals appropriately, and ensure that the chassis assembly did not exhibit mechanical binding or asymmetric friction. The diagnostic sequence was particularly valuable for identifying wiring errors introduced during assembly or repairs, as incorrect motor polarity could be immediately detected and corrected before autonomous operation commenced.

Mode 2, Full Autonomous mode, represented the competition-ready operational state in which the robot executed its complete mission profile without human intervention. Upon entering this mode, the system initiated the primary navigation loop, following the black line while monitoring for node detection events. The autonomous sequence progressed through six predefined waypoints, each requiring specific actions ranging from colour-sorted pallet deployment to timed forward advances for obstacle negotiation. The mode remained active until the final parking node was reached, at which point the robot executed a timed advance into the parking zone and issued a complete motor halt.

Transitions between modes were managed exclusively through the external interrupt service routine associated with the push-button on RB0/INT0. The ISR implemented a software debounce mechanism to filter mechanical switch bounce, preventing spurious mode

transitions. Upon detecting a valid button press, the ISR incremented the global `robot_mode` variable in a modular fashion, wrapping from Mode 2 back to Mode 0 to create a cyclic state progression. Critically, the ISR also executed an immediate safety stop command, disabling all motor outputs regardless of the previous operational state. This safety-first approach ensured that mode transitions could not result in uncontrolled motion, a vital consideration when operating in close proximity to competition officials and other participants.

Prior to entering any operational mode, the system executed a Power-On Self Test (POST) sequence during microcontroller initialization. The POST routine performed a visual LED chase pattern to verify basic I/O functionality, swept the servo mechanism through its full range of motion to confirm PWM generation, and sampled all five line sensors to detect potential open-circuit failures. Any anomalies detected during POST were indicated through a distinctive error pattern, alerting operators to hardware faults before competition commencement.

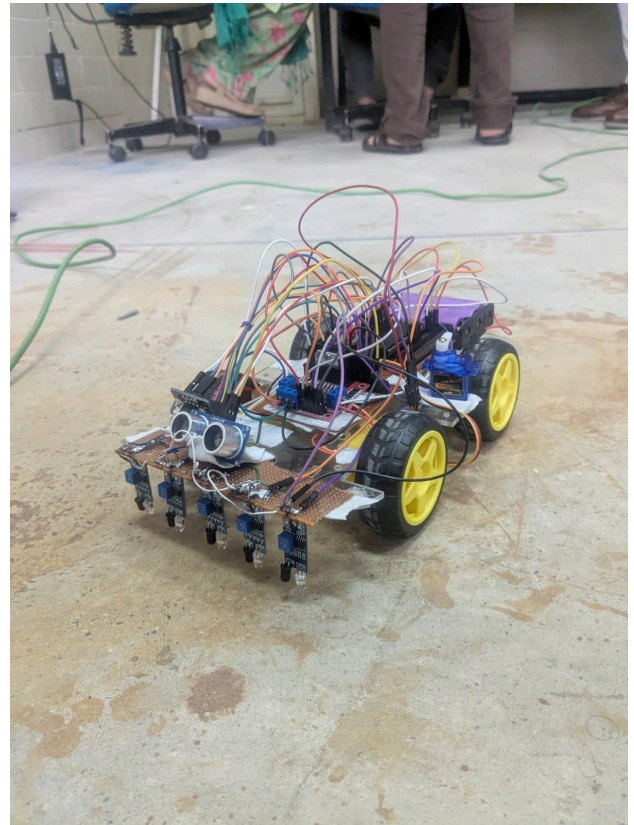
IV. HARDWARE DESIGN

A. Microcontroller

The PIC18F4550 was selected as the system microcontroller due to its availability in the institutional laboratory, adequate peripheral set for the application requirements, and extensive documentation. The device was configured to operate using its internal oscillator at 8 MHz, eliminating the need for external crystal components and reducing board complexity. Clock configuration was established through the `CONFIG1L` and `CONFIG1H` registers during device programming.

To maximize the number of available digital I/O pins, the analog-to-digital converter module was disabled by setting `ADCON1` to `0x0F`. This configuration converted all pins on `PORTA`, which default to analog inputs at reset, into standard digital I/O capable of interfacing with the TCRT5000 sensor array. Port direction registers were configured according to the signal flow requirements: `TRISA` was set to `0xFF` to configure `PORTA` as input for the five sensor channels, `TRISB` was set to `0x07` to

configure `RB0` through `RB2` as inputs (push-button and colour sensor) while `RB3` remained output for servo control, `TRISC` was set to `0x00` for digital outputs including colour sensor control, and `TRISD` was set to `0x00` to drive the four motor control signals to the H-bridge.



B. Motor Control

Differential drive was implemented using an L298N dual H-bridge motor driver integrated circuit, selected for its ability to handle the continuous current requirements of the DC gear motors and its built-in flyback diode protection against inductive kickback. The H-bridge received four control signals from `PORTD`: `RD4` and `RD5` controlled the left motor's forward and reverse inputs respectively, while `RD6` and `RD7` controlled the corresponding right motor channels. This arrangement provided independent control over each wheel's direction and enabled both differential steering and zero-radius turns.

The motor interface was abstracted through a unified drive function that accepted left and right motor states as

parameters. This abstraction layer accepted three discrete states per motor: forward, reverse, and brake. The brake condition was implemented by driving both H-bridge inputs high simultaneously, creating a dynamic braking effect that improved stopping performance compared to simple coasting. Six higher-level movement functions were built upon this abstraction: forward activated both motors in the forward state, stop engaged dynamic braking on both motors, *soft_left* reduced the left motor to brake while maintaining right motor forward motion to execute gentle curves, *soft_right* performed the mirror operation, *hard_left* reversed the left motor while advancing the right motor to achieve tight pivoting turns, and *hard_right* executed the corresponding mirror maneuver.

C. Sensor Array

Line detection was accomplished through five TCRT5000 reflective infrared sensor modules mounted at the front edge of the PCB. These sensors were positioned on PORTA with the following pin assignments: center sensor on RA2, inner-left on RA1, inner-right on RA3, outer-left on RA0, and outer-right on RA5. The physical spacing between adjacent sensors was maintained at 15 to 20 mm, a critical dimension determined through empirical testing to provide optimal line detection while remaining narrow enough to fit within the track's curvature constraints.

The TCRT5000 modules were configured to output active-low signals, meaning that a logic zero indicated detection of the dark line while logic one indicated the white background surface. This convention was maintained consistently throughout the software to avoid logical inversions that could introduce subtle bugs. Each sensor's digital output was protected by a 10 k Ω pull-up resistor to VDD, ensuring that disconnected or failed sensors would default to a safe state indicating no line detection.

The five-sensor configuration provided the necessary information density for implementing a priority-based line-following algorithm. The center sensor served as the highest-priority reference, indicating perfect alignment when active. The inner sensors triggered soft corrective turns, while the outer sensors initiated aggressive recovery maneuvers when the robot had drifted substantially off-course. This graduated response strategy

maintained smooth tracking during normal operation while providing strong corrective action when necessary.

D. Colour Detection

Pallet colour discrimination was implemented using a TCS3200 colour-to-frequency converter integrated circuit. This device contained an array of photodiodes with red, green, and blue filters, along with frequency-scaling circuitry that produced a square wave output whose frequency was proportional to the incident light intensity at the selected colour channel. The colour channel selection was controlled through two pins: S2 connected to RC2 and S3 connected to RB1. The frequency output appeared on the COLOR_IN pin connected to RB2.

For this application, a simplified colour discrimination approach was adopted rather than full frequency measurement. The system configured the TCS3200 to output the blue-filtered photodiode response, then sampled the digital state of COLOR_IN after a fixed settling period. The resulting binary classification distinguished blue pallets from red pallets with sufficient reliability for the competition environment's controlled lighting conditions. Blue detection commanded the servo to deploy the pallet at 90 degrees, while red detection triggered a 45-degree deployment angle, satisfying the competition's colour-sorting requirement.

E. Servo Mechanism

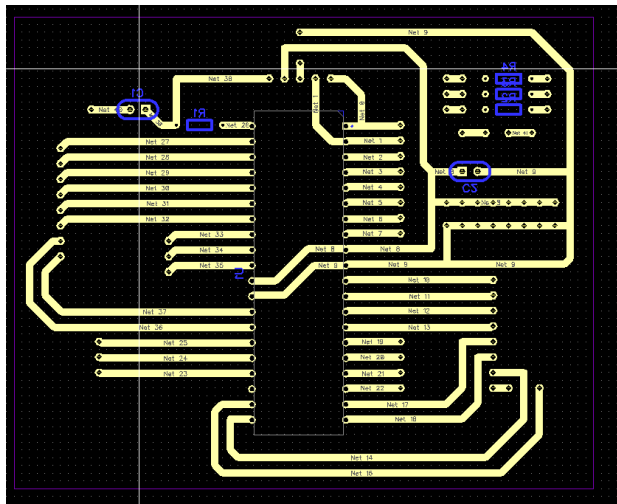
Pallet deployment was accomplished using a standard hobby servo motor controlled via software-generated pulse-width modulation on RB3. Standard hobby servos require a pulse train with a 20 ms period, where the pulse width determines the angular position: approximately 1 ms corresponds to 0 degrees, 1.5 ms to 90 degrees, and 2 ms to 180 degrees. The pulse width for a desired angle was calculated using the formula: $\text{pulse_width} = 1000 \mu\text{s} + (\text{angle} \times 1000/180)$.

To achieve stable positioning, the servo control routine generated forty consecutive pulses, each followed by a 20 ms delay to maintain the required update rate. This multi-pulse approach ensured that the servo mechanism had sufficient time to reach and settle at the commanded position before the pulse train ceased. The pulse

generation was implemented through precise busy-wait timing using the compiler's `__delay_us()` intrinsic function, which provided deterministic timing despite the lack of dedicated hardware PWM resources.

F. PCB Design

The complete system was integrated onto a custom printed circuit board designed using KiCad and fabricated through commercial PCB services. The board layout incorporated several critical design features to ensure reliable operation in the electrically noisy competition environment.



. The motor supply traces were routed with 50 mil widths to handle peak current demands during acceleration and locked-rotor conditions,

Decoupling capacitors rated at 100 nF ceramic were placed immediately adjacent to each VDD pin on the PIC18F4550, with trace lengths minimized to reduce parasitic inductance. Additional bulk capacitance of 100 μ F was positioned at the power input terminals to filter low-frequency supply variations. The five TCRT5000 sensor modules were surface-mounted at the front PCB edge with their 15 to 20 mm spacing enforced through precise footprint placement, ensuring consistent geometric alignment with the track surface.

V. SOFTWARE DESIGN

A. Architecture

The software architecture was structured as an interrupt-driven state machine implemented in a single `main.c` source file compiled with the XC8 compiler version 3.10 targeting the C99 language standard. The central architectural element was a volatile global variable named `robot_mode` that encoded the current operational state as an unsigned integer ranging from 0 to 2. The volatile qualifier was essential to inform the compiler that this variable could be modified asynchronously by the interrupt service routine, preventing optimization-related coherency issues.

The main execution loop consisted of a continuous `while(1)` structure that dispatched control to mode-specific handler functions based on the current value of `robot_mode`. This polling architecture was appropriate for the application because the main loop's latency requirements were relaxed compared to the mode-switching response, which was handled by the interrupt mechanism.

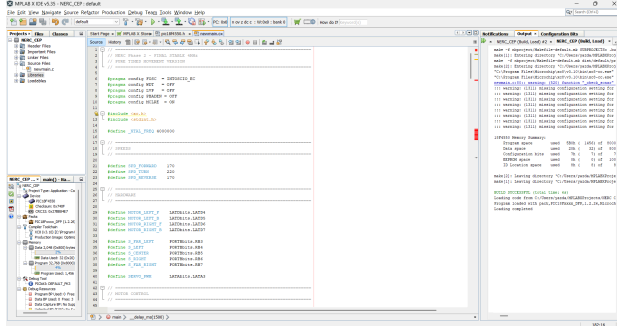
The external interrupt service routine associated with RB0/INT0 implemented three critical safety measures. First, a software debounce delay of 50 ms filtered mechanical switch bounce, preventing a single physical button press from registering as multiple mode transitions. Second, an unconditional safety stop command immediately disabled all motor outputs regardless of the previous state, ensuring that mode transitions could not result in uncontrolled motion. Third, state variables associated with the autonomous navigation sequence were reset, including node counters and navigation flags, preventing state persistence from interfering with subsequent autonomous runs.

B. Line Following Algorithm

The line-following algorithm implemented a priority-based decision tree that evaluated sensor states in a hierarchical manner to determine the appropriate corrective steering command. The algorithm read all five sensor values from PORTA and applied boolean logic to classify the robot's position relative to the line.

The highest priority condition was center sensor activation alone, indicating perfect alignment. In this state, the forward command maintained straight-ahead

motion. If the center sensor was not active, the algorithm proceeded to evaluate paired sensor combinations. Activation of the inner-left sensor (RA1) or both inner-left and center sensors triggered the `soft_left` maneuver, producing a gentle corrective curve. Similarly, inner-right activation initiated `soft_right` steering.



For more severe misalignment, the algorithm checked the outer sensors. Detection on the outer-left sensor (RA0) invoked `hard_left`, executing an aggressive pivot turn to recover the line. The outer-right sensor triggered the corresponding `hard_right` response. In the failure condition where no sensors detected the line, the algorithm commanded forward motion as a recovery behavior, under the assumption that the robot had momentarily lost the line due to surface irregularities or gaps but would reacquire it by continuing forward.

This priority structure ensured smooth tracking during normal operation while providing increasingly aggressive corrections as misalignment severity increased. The active-low sensor convention was maintained throughout the logic, with line detection represented by logic zero.

C. Node Detection

Course intersections, referred to as nodes, were identified by the simultaneous activation of all five sensors, indicating that the line sensor array was positioned over a crossbar marking. To prevent false triggers from electrical noise or momentary surface artifacts, node detection implemented a double-check debounce mechanism.

The `is_node()` function returned a boolean value indicating confirmed node presence. Upon detecting that all five sensors read logic zero, the function recorded this state, then executed a 70 ms delay and re-sampled the

sensors. Only if the all-active condition persisted through both samples did the function return true, confirming a valid node. This temporal filtering was crucial for competition reliability, as spurious node detections could cause premature navigation transitions and result in course failure.

D. Autonomous Navigation Sequence

The full autonomous mode implemented a six-node mission profile, with each node requiring specific actions. A global node counter incremented upon each confirmed node detection, serving as the primary navigation state variable.

Node 1 represented the first station, designated S1. Upon detection, the robot executed four pallet deployment operations using the `demo_placement()` function, then commanded a `hard_right` turn to align with the next track segment. Node 2 corresponded to station S2, requiring four additional placements followed by a `hard_left` turn. Node 3 marked the entry to a zigzag section characterized by sharp alternating curves. The robot executed a timed forward advance of 1000 ms to traverse this section, relying on the line-following algorithm to negotiate the curves without explicit zigzag logic.

Node 4 indicated the base of an inclined ramp section. A timed forward command of 1800 ms was issued to climb the ramp and reach the upper level, with the extended duration accounting for reduced velocity on the incline. Node 5 designated the third station, S3, where four final pallet placements occurred. Node 6 represented the parking zone entry, where the robot advanced 1200 ms into the marked area before issuing a full motor halt, completing the mission profile.

The timed forward advances at nodes 3, 4, and 6 represented a pragmatic engineering compromise. Encoder-based distance measurement would have provided more deterministic navigation, but was not implemented due to time and complexity constraints. The selected timing values were determined empirically during testing and provided acceptable reliability under consistent battery voltage conditions.

E. Colour-Sorted Pallet Deployment

The `demo_placement()` function encapsulated the complete pallet handling sequence. Upon invocation, the function first issued a stop command to halt robot motion and ensure stability during sensing and deployment. The colour sensor was then queried by configuring the TCS3200 to blue-filter mode and reading the `COLOR_IN` digital state after a 100 ms settling delay.

Based on the colour classification result, the servo was commanded to either 90 degrees for blue pallets or 45 degrees for red pallets. The servo control routine generated the appropriate pulse train for stable positioning at the target angle. After a 500 ms hold period to allow the pallet to disengage from the gripper mechanism, the servo returned to the 0-degree home position. LED feedback was provided throughout the sequence to indicate deployment status, aiding in visual monitoring during testing and competition runs.

VI. TESTING AND RESULTS

Verification of the robot's functionality proceeded through systematic testing of each operational mode. The POST sequence, executed automatically upon power application, provided immediate confirmation of basic hardware integrity. The LED chase pattern verified that the microcontroller's I/O ports were functioning and that the output circuitry was properly energized. The servo range test swept from 0 to 180 degrees in 45-degree increments, confirming that the software PWM generation was producing correct pulse widths and that the servo mechanism responded appropriately. The sensor bus check sampled all five TCRT5000 outputs and indicated any sensors reading a persistent zero or one state, which could indicate short circuits, open circuits, or improper sensor orientation.

Mode 0 testing demonstrated the value of the calibration state during competition preparation. By observing sensor indicator LEDs while manually positioning the robot over the line, operators could verify that the center sensor activated when properly aligned and that adjacent sensors responded as the robot was shifted laterally. This visual feedback enabled rapid mechanical adjustments without requiring code modifications or external test equipment.

Mode 1 diagnostic drive testing successfully identified several wiring errors during initial assembly. In one instance, the left motor's phase wires had been reversed, causing the robot to execute a tight spin when forward motion was commanded. The diagnostic sequence made this failure mode immediately obvious, allowing correction before attempting autonomous operation. Similarly, the turn tests revealed that one H-bridge input had an intermittent connection, causing erratic behavior during rotational maneuvers. This fault would have been extremely difficult to diagnose during autonomous mode but was trivially identified in the diagnostic sequence.

Mode 2 autonomous operation demonstrated reliable line following under laboratory conditions with consistent fluorescent lighting. The robot successfully tracked straight line segments, negotiated 90-degree corners using the priority-based algorithm's hard turn responses, and maintained stability through gentle curves using soft corrections. Node detection proved robust with the 70 ms double-check debounce, showing zero false triggers during repeated test runs while reliably identifying actual crossbar intersections.

The significance of power domain isolation became apparent during motor switching events. Initial prototypes without proper ground plane separation exhibited microcontroller resets during hard turn commands due to ground bounce coupling from the motor driver's high di/dt switching. Implementation of the flood-filled ground plane and strategic placement of decoupling capacitors eliminated these reset events, demonstrating the critical importance of proper power distribution network design in mixed-signal systems.

VII. CHALLENGES AND LIMITATIONS

Several technical challenges were encountered during development, some of which were successfully resolved while others remain as limitations of the current implementation. Ambient light interference on the IR sensors represented a persistent challenge, particularly under variable lighting conditions that might be encountered in different competition venues. The TCRT5000 sensors exhibited sensitivity to both direct sunlight and certain types of artificial lighting,

occasionally resulting in false line detections. While the double-check debounce on node detection helped mitigate this issue, no comprehensive solution for general lighting immunity was implemented. Future revisions could explore adding optical filters or implementing adaptive threshold adjustment based on ambient light sensing.

Motor switching noise posed significant electromagnetic compatibility challenges. The rapid current reversals during direction changes and the inductive kickback from motor windings generated electrical transients that coupled into sensitive analog portions of the sensor circuitry. This coupling was partially mitigated through the use of 100 nF ceramic decoupling capacitors placed immediately adjacent to the PIC18F4550's power pins and by implementing a solid ground plane on the bottom copper layer to provide low-impedance return paths. However, occasional sensor glitches were still observed during aggressive turning maneuvers, suggesting that additional filtering or shielding might be beneficial.

The reliance on timing-based navigation at nodes 3, 4, and 6 represented a fundamental architectural limitation. Without encoder feedback on the drive motors, distance traveled was estimated purely through timed forward commands. This approach exhibited sensitivity to battery voltage variations, as motor speed decreased as the battery discharged, leading to shorter travel distances for identical timing values. While the selected timing values provided acceptable reliability during testing with fresh batteries, performance degradation toward the end of competition days could pose issues. Implementation of quadrature encoders and closed-loop distance control would significantly improve navigation determinism.

The TCS3200 colour sensor's frequency output was not utilized to its full potential in the current implementation. Rather than measuring the actual output frequency, which would have provided quantitative colour intensity data, the system simply sampled the digital state of the output pin after a fixed delay. This binary classification approach proved adequate for distinguishing between clearly different colours under controlled lighting but lacked the robustness that full frequency measurement would provide. Implementing a proper frequency measurement routine using the microcontroller's timer modules would enable more sophisticated colour

discrimination and potentially allow for white balance calibration.

A significant software limitation arose from the blocking nature of the `__delay_ms()` function used throughout the code, particularly during servo pulse generation. When executing the 40-pulse sequence required for stable servo positioning, the main execution thread was completely blocked for approximately 800 ms. During this period, sensor sampling was suspended, meaning that if the robot drifted off the line during a deployment operation, no corrective action could be taken until the servo sequence completed. This issue could be addressed by implementing a non-blocking delay mechanism using timer interrupts or by utilizing the PIC18F4550's hardware PWM capabilities through the CCP module.

VIII. CONCLUSION

positions as a continuous error signal, generating proportional steering corrections rather than the current bang-bang control approach. Second, integration of quadrature encoders on the drive motors would enable closed-loop distance control, eliminating the battery voltage sensitivity inherent in timing-based navigation. Encoder feedback would also enable precise turn angle control, improving node-to-node navigation accuracy. Third, migration of the servo control to the hardware PWM module via the CCP peripheral would eliminate the blocking delays currently required for software pulse generation, allowing concurrent sensor sampling during pallet deployment operations and improving system responsiveness.

The project demonstrated the successful application of embedded systems principles to a complex real-world robotics challenge, validating both the hardware design decisions and the software architectural choices through systematic testing and competition-relevant operation. This project successfully designed, implemented, and tested an autonomous line-following robot capable of competing in the NERC/CEP 2026 Autonomous Robot Challenge. The

interrupt-driven state machine architecture proved highly effective for managing the three operational modes, enabling systematic testing and debugging while providing instantaneous safety-critical mode transitions. The robot demonstrated reliable line tracking through a priority-based sensor algorithm, accurate node detection via temporal debouncing, and successful colour-discriminated pallet deployment using the TCS3200 sensor and servo mechanism.

The modular software architecture and comprehensive diagnostic modes significantly reduced development time by isolating testing to specific subsystems. Mode 0 calibration enabled rapid sensor alignment verification, Mode 1 diagnostic drive quickly identified motor wiring errors, and Mode 2 autonomous operation executed the complete mission profile. The hardware design's emphasis on power domain isolation and strategic decoupling proved essential for reliable operation in the electrically noisy environment created by motor switching events.

Several specific improvements would enhance future revisions of this design. First, implementation of PID-based line following would provide smoother tracking and better disturbance rejection compared to the current discrete steering commands. The PID controller could utilize a weighted sum of sensor

[5] ams-OSRAM AG, "TCS3200, TCS3210: Programmable Color Light-to-Frequency Converter," Document Number 000214, Rev. C, 2012.

[6] J. W. Valvano, *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers*, 5th ed. Austin, TX: CreateSpace Independent Publishing, 2012.

REFERENCES

[1] Microchip Technology Inc., "PIC18F2455/2550/4455/4550 Data Sheet: 28/40/44-Pin, High-Performance, Enhanced Flash, USB Microcontrollers with nanoWatt Technology," DS39632E, 2009.

[2] Microchip Technology Inc., "MPLAB XC8 C Compiler User's Guide for PIC," DS50002053G, 2020.

[3] STMicroelectronics, "L298 Dual Full-Bridge Driver," DocID2388 Rev 9, 2000.

[4] Vishay Semiconductors, "Reflective Optical Sensor with Transistor Output – TCRT5000, TCRT5000L," Document Number 83751, Rev. 1.3, 2017.